

Kubernetes Services & Ingress — Reference Notes

Personal study notes — built hands-on with Minikube (Docker driver) on macOS, cross-checked against Docker Desktop Kubernetes.

1. Core building blocks

Object	Purpose	Self-healing?
Pod	Smallest deployable unit — one or more containers	No
ReplicaSet	Keeps N copies of a Pod template running	Yes (via reconcile loop)
Deployment	Manages ReplicaSets, adds rolling updates/rollback	Yes — the one you use day to day

Chain: Deployment → creates ReplicaSet (using `selector` + `template`) → ReplicaSet creates Pods → keeps count at `replicas`.

Generate manifests via CLI (don't hand-write from scratch)

```
kubectl run nginx-pod --image=nginx --dry-run=client -o yaml > pod.yaml

kubectl create deployment nginx-deploy --image=nginx --replicas=3 \
  --dry-run=client -o yaml > deploy.yaml

kubectl apply -f pod.yaml
kubectl apply -f deploy.yaml
kubectl get pods,rs,deploy -o wide
```

Deployment YAML anatomy

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3                                # "create N copies"
  selector:
    matchLabels:
      app: nginx-deploy                     # QUERY: "find pods with this label"
  template:                                  # a full Pod spec, embedded
    metadata:
      labels:
        app: nginx-deploy                   # STAMP: label put on every pod created
    spec:
      containers:
        - image: nginx
          name: nginx
```

`selector` and `template.metadata.labels` must match — if they don't, `kubectl apply` rejects the Deployment outright. Kubernetes also auto-adds a `pod-template-hash` label on top of your own label, so rolling updates can distinguish "old version" pods from "new version" pods (each version gets its own ReplicaSet).

Proving self-healing

```
kubectl delete pod <one-of-the-three-pods>
kubectl get pods -o wide # a brand-new pod appears, new name + new IP, count back to 3
```

A bare Pod (no controller) does **not** come back if deleted — only ReplicaSet/Deployment-managed pods self-heal.

2. Service IP types

Type	What it is	EXTERNAL-IP	Reachable from
ClusterIP (default)	Stable virtual IP for a Service	<none>	Inside cluster only
NodePort	ClusterIP + a port (30000-32767) opened on every node's IP	<none>	Anyone reaching a node IP
LoadBalancer	NodePort + cloud provider assigns a real external IP	real IP (cloud) / faked locally	Public internet (real cloud)
ExternalName	Pure DNS CNAME, no IP, no selector, no pods	n/a	Just a DNS redirect to an external host

Why Services exist at all

Pod IPs change every time a pod is recreated (self-healing, rolling update, etc). A Service gives a stable IP/DNS name that always routes to whichever pods currently match its label `selector` — same wiring mechanism as Deployment → ReplicaSet → Pod, just applied to Service → Pod.

port vs targetPort vs nodePort

```
client --curl on nodePort or LB port--> [ Service ] --forwards to targetPort--> [ Pod's container port ]
```

- `port` — the port the **Service itself** listens on, inside the cluster (ClusterIP address book entry).
- `targetPort` — the port on the **actual container** traffic gets forwarded to. Must match what the process really listens on — if wrong, you get `Connection reset by peer`, not a helpful error.
- `nodePort` — an extra doorway opened on **every node's own IP** (not "the cluster" abstractly), only for NodePort/LoadBalancer types. Same port is open identically on every node, even ones with no matching pods.

```
NAME          TYPE        PORT(S)
nginx-nodeport NodePort    80:30740/TCP
               ^  ^
               port nodePort (targetPort=80 too, hidden since it equals port here)
```

Creating each type

```
# ClusterIP (default - omit --type)
kubectl expose deployment nginx-deploy --port=80 --target-port=80

# NodePort
kubectl expose deployment nginx-deploy --port=80 --target-port=80 \
  --type=NodePort --name=nginx-nodeport

# LoadBalancer
kubectl expose deployment nginx-deploy --port=80 --type=LoadBalancer --name=nginx-lb
```

3. Accessing each type locally – the messy part

This differs a lot by local tool, and mismatching them is a common source of confusion.

Minikube (Docker driver on macOS)

Type	How to actually reach it
ClusterIP	Only from inside the cluster: <code>kubectl run tmp --rm -it --image=busybox --restart=Never -- sh</code> then <code>wget -O- <clusterIP></code> or <code>wget -O- <svc-dns-name></code>
NodePort	<code>minikube ip</code> is not directly reachable from macOS host on Docker driver — use <code>minikube service <name> --url</code> instead, which prints a tunneled <code>127.0.0.1:PORT</code> URL
LoadBalancer	Stays <code><pending></code> forever without help. Run <code>sudo minikube tunnel</code> in a separate terminal (keep it open) — it assigns a real routable IP (often <code>127.0.0.1</code>) and forwards traffic
Any type — universal fallback	<code>kubectl port-forward svc/<name> 8081:80</code> then <code>curl localhost:8081</code> . Works regardless of driver/tool quirks.

Gotcha proven hands-on: if `minikube tunnel` is already bound to port 80 for one LoadBalancer service (e.g. `nginx-lb`), a second thing wanting port 80 (like the ingress controller) can silently lose that port — requests land on the wrong backend with no obvious error, just a confusing wrong response. Delete unused LoadBalancer services when debugging routing.

`minikube service` / `minikube tunnel` always target the Minikube profile directly — they ignore your current `kubectl` context entirely.

Docker Desktop Kubernetes

Type	How to actually reach it
LoadBalancer	Docker Desktop ships its own built-in LB controller: assigns a Docker-internal IP (e.g. <code>172.18.0.5</code> , not reachable directly from the Mac) <i>and separately</i> auto-forwards that port straight to <code>localhost</code> . So just <code>curl localhost:<port></code> works, no tunnel.
NodePort (plain)	No automatic forwarding — the node's internal IP (e.g. <code>172.18.0.4</code>) is unreachable from the host, same as Minikube's problem. Use <code>kubectl port-forward</code> instead.

Verified by testing: `curl 172.18.0.5:80` (LB's own "external" IP) hangs even on Docker Desktop — only the `localhost` auto-forward works. Don't assume "Docker Desktop forwards everything to localhost" — it only does this specifically for LoadBalancer-type Services.

Real cloud clusters (GKE/EKS/AKS)

EXTERNAL-IP is a genuinely public, routable IP from a real cloud load balancer — curl it directly, no tunnel/port-forward/localhost tricks needed.

4. Ingress

Why it exists

Without Ingress: every externally-facing app needs its own NodePort (random ugly port) or its own LoadBalancer (a whole separate IP/cost in real cloud). With 10 apps that's 10 ports or 10 IPs to manage.

Ingress: **one** entry point (one controller, one IP), with routing rules based on **path** or **hostname** deciding which internal ClusterIP Service receives the traffic.

Ingress is **only routing rules** — it does nothing by itself. You need an **Ingress Controller** (real running software, e.g. nginx-ingress) installed to actually read and enforce those rules.

Enable on Minikube

```
minikube addons enable ingress
kubectl get pods -n ingress-nginx
kubectl get svc -n ingress-nginx # controller Service is type NodePort on Minikube
```

Controller lives in its own `ingress-nginx` namespace — it's cluster-wide infrastructure, not tied to one app.

Ingress rules point at Service *names*, never at IPs. In `--rule="/nginx=nginx-clusterip:80"`, `nginx-clusterip` must exactly match an existing Service's `metadata.name` in the same namespace — it is not an IP you supply.

Chain: Ingress rule → Service NAME + port → Kubernetes resolves the Service's Endpoints → real Pod IPs. This is visible in `kubectl describe ingress` output: `nginx-clusterip:80 (10.244.0.23:80,10.244.0.22:80,...)` — the name you typed on the left, the Pod IPs Kubernetes resolved it to on the right, in parentheses. A typo'd or missing Service name still lets the Ingress get created, but that path resolves to nothing when traffic hits it.

Path-based routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: demo-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - http:
      paths:
      - path: /nginx
        pathType: Prefix
        backend:
          service: { name: nginx-clusterip, port: { number: 80 } }
```

```
- path: /hello
  pathType: Prefix
  backend:
    service: { name: hello-clusterip, port: { number: 80 } }
```

Critical gotcha, proven hands-on: Ingress does **not** strip the matched path before forwarding to the backend by default. A request to `/nginx` is forwarded to the backend pod as literally `/nginx` — if the pod only serves `/`, you get a 404 *from the backend itself* (note: the 404 page shows the backend's own server version, e.g. `nginx/1.31.6` — proof the routing worked, just the path was wrong).

Fix: add annotation `nginx.ingress.kubernetes.io/rewrite-target: /` and use `pathType: Prefix` (not `Exact` — `Exact` requires character-for-character match, no trailing-slash flexibility).

```
kubectl create ingress demo-ingress \
  --rule="/nginx=nginx-clusterip:80" \
  --rule="/hello=hello-clusterip:80" \
  --dry-run=client -o yaml > ingress.yaml
# then add the rewrite-target annotation manually, apply
kubectl apply -f ingress.yaml
```

Host-based routing

```
kubectl create ingress host-ingress \
  --rule="nginx.local/*=nginx-clusterip:80" \
  --rule="hello.local/*=hello-clusterip:80" \
  --dry-run=client -o yaml > host-ingress.yaml
```

Generated rule structure gains a `host:` field as a sibling of `http:` — this is what lets the exact same IP/port serve completely different backends based on the incoming `Host` HTTP header.

Since `nginx.local` / `hello.local` aren't real DNS names, map them locally in `/etc/hosts` (checked before any real DNS lookup):

```
sudo nano /etc/hosts
# add:
127.0.0.1 nginx.local hello.local
```

```
curl http://nginx.local/
curl http://hello.local/
```

Full request path — the piece that resolves "how does this reach a ClusterIP that's normally unreachable"

```
Your Mac (curl http://nginx.local/)
  |
  v
127.0.0.1:80      <-- /etc/hosts maps nginx.local -> 127.0.0.1
  |
  v
minikube tunnel   <-- process actually listening on 127.0.0.1:80,
  |               forwards into the Minikube VM's network
```

```

      v
ingress-nginx-controller Service (NodePort, ns: ingress-nginx)
      |
      v
ingress-nginx-controller POD <-- reads Ingress rules, inspects
      |                               Host header / path, decides target
      v
nginx-clusterip Service (ClusterIP) <-- reached from INSIDE the cluster,
      |                               by the controller pod -- never from your Mac
      v
nginx-clusterip Pod                <-- final destination

```

Key insight: your Mac only ever talks to one thing — the ingress controller's entry point. The controller pod runs *inside* the cluster, so its calls to ClusterIP Services are normal internal traffic. This is the entire value of Ingress: one external bridge point for many internal Services, instead of NodePort/LoadBalancer per app.

5. Debugging habits picked up along the way

- **Always check `kubectl config current-context`** before assuming "my resource is missing" — `docker-desktop`, `minikube`, and any real cluster are completely separate state. Empty `kubectl get svc` often means wrong context/namespace, not a real problem.
- **`metadata.name` is immutable** — `kubectl edit` to rename an object fails outright ("At least one of apiVersion, kind and name was changed"). To rename: `kubectl delete` the old object, then `kubectl apply` a manifest with the new name (the ReplicaSet's `pod-template-hash` can end up identical if the pod template content is unchanged, since the hash is content-based, not time-based).
- **Filename ≠ object identity** — Kubernetes only reads `metadata.name` inside the file; a file called `nginx-deploy.yaml` can quietly contain `name: nginx-clusterip` after an edit, causing confusing `kubectl apply -f .` output later. Keep filenames in sync with the object name inside.
- **`kubectl apply -f .` applies every YAML in the directory** — safe only if you know exactly what's in there; stray/experimental files get silently (re)applied too.
- **Port already in use locally** (e.g. `kubectl port-forward` failing on 8080) — check with `lsof -i :8080` before killing anything, to see what's actually holding it.
- **Two Services fighting over the same external port** (e.g. two `LoadBalancer`-type Services both wanting port 80 via `minikube tunnel`) causes traffic to silently land on the wrong backend — always check for this when a Service returns unexpected content.
- **`kubectl describe ingress <name>`** shows resolved backend Pod IPs per rule — proves whether Ingress is actually matching/routing correctly, independent of whatever the final HTTP response looks like.